Research Foundation

Campus Equipment Checkout System

Raghuram Kalasani | COMP-699-A | Week 1 | October 18, 2026

Status and connection to Step 1

This is a proposed research refinement for student and instructor review, not a claim that a new research question has already been approved. The available Step 1 planning documents specify FR-06 (reject overlapping approvals) and NFR-02 (atomicity and concurrent integrity), but do not state a formal primary research question. This refinement makes those requirements experimentally measurable. Earlier submissions remain unchanged.

Primary research question

For a local Django/SQLite equipment-reservation prototype, how does transaction-protected approval compare with a check-then-save implementation without a shared transaction in preventing overlapping approvals under concurrent requests? What retry and latency tradeoffs occur under the tested workloads?

Working hypothesis

Keeping the conflict check and status update in one immediate transaction is expected to prevent overlapping approvals in the controlled workloads, while a separate check and update may admit a race. Protection may increase waiting or retry outcomes. This is a testable expectation, not an established result; a baseline that shows no violations in a run does not establish its safety.

Research chain

Question: concurrent booking integrity. Inputs: reproducible synthetic reservation intervals and controlled worker settings. Method: paired runs of two approval implementations on equivalent fresh databases. Evidence: independently audited approved-record overlaps, approval outcomes, retries, errors and per-attempt duration. Claim: a bounded statement about observed behavior under documented local conditions.

Scope of this milestone

Week 1 implements input generation, validation, CSV round-trip checking and descriptive overlap counts. It does not implement or execute the comparison, run concurrent approvals, measure latency, or establish the hypothesis. The earlier approval proof of concept and inventory web application provide background, but their results are not substituted for new research experiments.

Inputs and Computational Environment

Research inputs and provenance

All inputs are generated locally; there are no downloaded datasets, external API calls, real borrowers or university records. The default generator uses seed 699, 200 requests and 10 equipment IDs. Each request has a unique integer ID, an equipment ID sampled uniformly with replacement, a start selected from 0 through 465 minutes in 15-minute increments, and a duration selected uniformly from 15, 30, 60 or 120 minutes. Realized counts need not be uniform. End times can extend beyond the 480-minute start window.

Time and eligibility assumptions

Minutes are offsets from an experimental origin, not wall-clock timestamps. Intervals are start-inclusive and end-exclusive; touching endpoints do not overlap. Week 2 should map offsets to a controlled future origin or inject a fixed clock to keep eligibility repeatable. Equipment is assumed active and requests pending when inserted for the experiment. Arrival order and worker concurrency are separate experimental variables and are not represented by the reservation start time.

Schema and validation

requests.csv has request_id, equipment_id, start_minute and end_minute. Validation rejects missing or unexpected columns, non-integer values, nonpositive or duplicate request IDs, unknown equipment IDs, negative starts, and ends not greater than starts. The program reloads the CSV, validates it again and checks exact equality to the generated records. Identical intervals with distinct IDs are legitimate competing requests, not duplicate-record errors.

Environment and organization

Recorded execution: Python 3.12.14, macOS/Darwin arm64. The Week 1 scripts use the Python standard library only. src/research/prepare_inputs.py contains generation, validation, overlap counting and the command-line runner. src/research/test_prepare_inputs.py contains six validation tests. src/research/requirements.txt documents the absence of third-party dependencies. Documentation and captured outputs are under docs/step_2/week_1/.

Relationship to the application

The existing Django application and reservation proof of concept remain separate and unchanged. Week 1 does not open their database or import their models. Week 2 should adapt the tested approval logic into the experiment harness and record the actual Django/SQLite versions, database configuration and source revision used for each run.

Execution and Preliminary Output

Reproduce from the package or repository root

Use Python 3.12, preferably the same patch version as the recorded run. No pip installation is needed for these scripts. Run the two commands below. Replace python3 with the path to your Python 3.12 interpreter if necessary. The output option identifies a directory whose generated files will be replaced on rerun.

Commands

python3 src/research/prepare_inputs.py --count 200 --equipment 10 --seed 699 --output docs/step_2/week_1/evidence
python3 -m unittest discover -s src/research -p "test_*.py" -v

Observed results

The recorded run produced 200 valid requests using all 10 equipment IDs, with 13 to 25 requests per item. Among 1,962 unordered pairs sharing an equipment ID, 414 had overlapping time intervals (21.10%). Durations were 15 minutes for 40 requests, 30 for 49, 60 for 52, and 120 for 59. CSV reload and validation passed. These are descriptive properties of generated inputs; no requests were approved by this script.

Validation evidence

All six test methods passed. They cover deterministic seeds and seed variation, generated constraints, invalid records, both endpoint boundaries and several overlap shapes, a manually counted four-record example, and invalid generation parameters. The four-record example has three same-equipment pairs and exactly one overlap. Tests check input preparation rather than database transaction correctness.

Evidence files and integrity

evidence/requests.csv contains inputs; summary.json contains parameters, environment, validation and computed summaries; run_output.txt captures the preparation output; test_output.txt captures the unittest run. CSV SHA-256: b5eb53e3ff1287c47019651baf823a7a81b0090de36064aa2bcd8552c19ec5d8. Repeating the same parameters in the recorded Python environment should reproduce the CSV; output paths, environment metadata and test timing may differ.

Execution attribution

The assistant executed these commands in the local workspace on October 18, 2026 and prepared the code and report with AI assistance. The student independently repeated input generation and the six tests on October 18, 2026 using Python 3.9.6 on Darwin arm64. The saved student_run/summary.json and test_output.txt confirm validation and six passing tests (0.004 seconds). The student CSV is byte-for-byte identical to the assistant CSV, with the same SHA-256. This demonstrates reproduction of input preparation across these two Python versions, not validation of the planned concurrency experiment. AI assistance remains disclosed; the student should review and explain the code before submission.

Limitations and Week 2 Plan

Early methodological issues

The existing software requirements were broader than a research experiment. This milestone narrows the proposed investigation to concurrent reservation integrity, rather than asserting that building an admin interface answers a research question. Synthetic workloads permit controlled repetition but cannot establish real campus demand, user satisfaction, time savings or production capacity. No empirical arrival distribution is available.

Measurement limitations

Input overlap counts are potential conflicts, not approval errors. Pair counts share requests and are not independent observations. Random duration and start choices induce overlap indirectly; a requested overlap rate is not controlled by this generator. Pair counting is quadratic and suitable for this small input sample; larger studies may need a sweep-line or indexed method. A seed fixes input generation, not operating-system thread scheduling.

Week 2 implementation

1. Build a harness with isolated temporary file-backed databases and independent worker connections. Adapt the existing protected approval logic; implement a clearly labeled experimental check-then-save baseline with the same eligibility rules.
2. Add a post-run audit of approved same-item intervals, independent of the approval query. Validate the audit with known valid and deliberately conflicting records.
3. Use identical workloads for both methods and reset each database before each trial. Record worker count, timeout, journal mode, seed, attempt order and environment. Start with one worker as a sequential reference and two workers for contention. Any forced race schedule must be separately labeled as a stress test.
4. Record approval, conflict, retry and unexpected-error counts; audit overlap violations and measure per-attempt elapsed time. Do not hide retries in successful-attempt averages.
5. Execute initial comparisons and report unexpected results and limitations in method_and_baseline.pdf.

Planned metrics and interpretation

The primary integrity measure is the number of unordered overlapping approved pairs for the same equipment after each trial; also report the fraction of trials with any violation. Secondary measures are outcome counts and median and 95th-percentile per-attempt latency, with units and percentile convention documented. Zero observed violations supports only the tested conditions. Week 3 should expand seeds and workload/concurrency settings and discuss variability.

Progress and submission

No Week 2 or Week 3 results are claimed. This package has not been pushed to GitHub or submitted to Canvas by the assistant. Place this report and evidence under docs/step_2/week_1/ and code under src/research/. Commit real changes as they occur; do not invent or backdate weekly progress. Confirm the proposed question against the approved research design before submission.

Local source basis

Step 1: 01_problem_scope_requirements.md, FR-06 and NFR-02; 02_architecture_and_data_design.md; existing src/reservation_poc.py. These are project materials, not independent scholarly evidence. This foundation does not claim a literature review or replace the scholarly sources from earlier coursework.